Biological Data Project Report

Marco Venanzi, Beatrice Riello and Francesco Bortolozzo

University of Padova

Academic Year 2025/2026

Abstract—We formulate Protein Function Prediction as a multi-label classification problem: given proteins identified by an ID and described by numerical features (embeddings), optionally enriched with InterPro domain information, we predict the set of Gene Ontology (GO) terms associated with each protein. We treat Cellular Component (CC), Molecular Function (MF), and Biological Process (BP) as separate tasks to keep label spaces coherent and manageable. The pipeline loads heterogeneous training resources, aggregates GO labels per protein and per sub-ontology, analyzes label frequency distributions, builds feature/target matrices, and trains three wide neural networks under early stopping. To avoid optimistic evaluation, we preserve a held-out test set and calibrate the probability-to-label threshold on validation by maximizing macro-F1. We discuss why macro-F1 was selected for model selection under strong class imbalance and how AUPR / $F_{\max}$ relate to CAFA evaluation.

Project materials. All code, notebooks, and outputs for this project are available in the following shared folder: Google Drive repository.

1. OBJECTIVE AND SCOPE

Protein Function Prediction (PFP) is naturally **multi-label**: a single protein can belong to multiple functional categories simultaneously. Our output is therefore a *set* of GO terms rather than a single class label. In practice, we predict a probability for each GO term and then transform those probabilities into a discrete annotation set.

We model proteins using

- **HDF5 embeddings (baseline):** fixed-length vectors that make learning straightforward and scalable.
- **InterPro domains (optional enhancement):** biologically interpretable features that may complement embeddings.

1.1. Why do we split the task into CC / MF / BP

GO is composed of three namespaces: **Cellular Component (CC)**, **Molecular Function (MF)**, and **Biological Process (BP)**. We treat them as three separate prediction problems. The underlying reasoning is pragmatic: each namespace has different label counts, granularity, and imbalance patterns. A single unified output layer would be unnecessarily large and would make debugging much harder, because poor performance could come from one namespace dominating the optimization.

Retrospective

As required by the assignment we used three focused models over one giant classifier: smaller and more coherent label spaces reduce accidental coupling between unrelated terms and make mistakes easier to interpret.

2. DATA SOURCES AND INGESTION (WHAT WAS LOADED AND WHY)

The pipeline assembles training data by merging multiple files around a single stable key: the `protein ID`. This "single key" principle is important in biological pipelines because silent ID mismatches are one of the most frequent sources of incorrect training sets.

2.1. GO annotation table: `train_set.tsv`

We load `train\_set.tsv` and immediately normalize column names (e.g., `Protein\_ID` → `ID`, `aspect` → `sub\_ontology`). It reduces the chance of future merge errors.

2.2. ID consistency check: `train_ids.txt`

We load `train\_ids.txt` and compare its length to the number of unique IDs in `train\_set.tsv`. The "hidden thought" is defensive programming: if IDs disagree, everything downstream (embeddings, domains, labels) may be misaligned even if code runs without errors.

2.3. FASTA sequences: inspected but not used

We also have `train.fasta`. We inspected the FASTA structure (ID vs description vs name) and checked whether sequence *features* were encoded in the file. Since the modeling input was defined as embeddings (already derived from sequence) and FASTA did not provide additional numerical features aligned with our scope, we deliberately did not use FASTA in the final feature matrix. (We therefore removed it from the notebook).

Retrospective

FASTA is biologically central, but using it directly would have required a different modeling approach (sequence models or handcrafted features). Embeddings gave us a strong baseline while keeping the project feasible.

2.4. Embeddings: `train_embeddings.h5`

Embeddings are stored in an HDF5 file where each dataset entry corresponds to a protein ID. We iterate over HDF5 keys and build a two-column table `(ID, embeddings)`. This representation is convenient because it keeps ID alignment explicit and makes it simple to stack vectors into a matrix later.

2.5. InterPro mapping: `train_protein2ipr.dat`

We load the InterPro mapping and rename columns for clarity (`ID`, `ipr`, `domain`, `familyID`, `start`, `end`). Then we group by `ID` and aggregate the remaining columns into tuples. The goal is to associate each protein with a compact list/tuple of domains (and related metadata) that can later be transformed into features.

Why grouping early helps. If we keep one row per domain occurrence, merges with the embedding table would duplicate proteins and corrupt the learning matrix. Grouping prevents that by enforcing one row per protein before merging.

2.6. GO structure: `go-basic.obo`

We parse `go-basic.obo` to obtain term metadata (IDs, namespaces) and relationships (`is\_a`, and when present `part\_of`). Even if the predictive model is a flat classifier, understanding the DAG structure matters for interpretation and for CAFA-inspired evaluation, which uses ontology-aware processing such as ancestor closure and consistent subgraphs.

3. TRAINING SET CONSTRUCTION (FROM RAW TABLES TO CC/MF/BP DATASETS)

3.1. Aggregating GO terms per protein and per sub-ontology

The raw GO table contains one row per `(protein, GO\_term, aspect)`. For multi-label learning we need one row per protein. We therefore

aggregate GO terms by ID within each sub-ontology:

$$\text{protein } i \longmapsto (t_{i1}, t_{i2}, \dots, t_{ik_i}).$$

This produces three target tables, one for each namespace (CC/MF/BP).

Why aggregation is non-negotiable. Without aggregation, the same protein would appear multiple times with different single labels. That would incorrectly turn multi-label learning into single-label learning with duplicated input.

3.2. Merging embeddings and (optionally) InterPro with labels

We create a combined feature table by merging:

- embeddings table (`ID, embeddings`)
- InterPro grouped table (`ID, ipr/domain/...`).

The InterPro merge is performed with `how='left'` so that the proteins remain present even if InterPro is missing. We explicitly count missing InterPro rows as a sanity check.

Why we later merge with `how='right'` into each ontology dataset. When building CC/MF/BP datasets, labels are the "must-have" (we cannot train without targets). We therefore right-join features into the aggregated label table: every labeled protein is kept, and any missing features are exposed as missingness to be handled.

Retrospective

We treated labels as the anchor: it is better to explicitly see that some labeled proteins miss optional features, than to silently drop labeled proteins and reduce training coverage.

3.3. Label distribution and term-frequency analysis

To quantify the imbalance of the label space in each ontology, we computed term frequencies after aggregating GO annotations per protein. For each namespace (BP, CC, MF), we show: (i) the *top 50* most frequent GO terms.

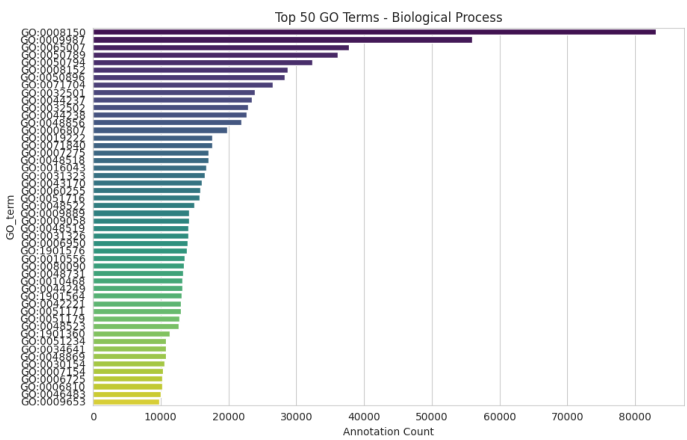


Figure 1. Biological Process (BP): top-50 most frequent terms.

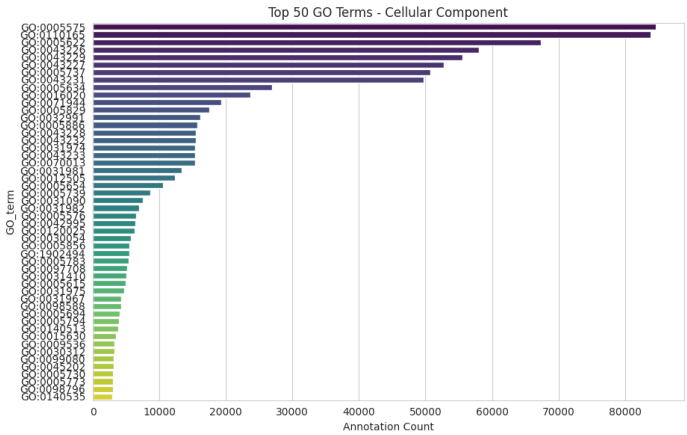


Figure 2. Cellular Component (CC): top-50 most frequent terms.

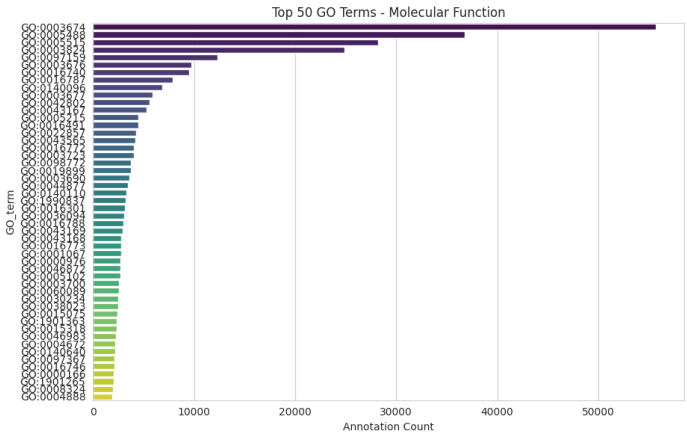


Figure 3. Molecular Function (MF): top-50 most frequent terms.

3.4. The white-noise

From a pure ML viewpoint, a term present in ~100% of samples carries little discriminative power and can be viewed as "white noise": it increases output dimensionality and training cost while being hard to learn meaningfully.

However, for GO tasks the semantic context matters. CAFA-inspired pipelines and ontology-aware interpretations frequently reason with ancestor closure/consistency. Removing very general terms may look clean from a classification standpoint, but it can also lead to systematically missing broad parent-level annotations.

Retrospective

We accepted some inefficiency to preserve GO completeness: even if some labels are weakly informative, dropping them could bias the annotation set toward overly specific terms and harm semantic consistency.

About filtering code. We implemented filtering logic (remove too-rare terms, that were below the advertised threshold, and optionally too-frequent terms), but we treated it as an optional experiment. The final training matrices used the complete label sets, consistent with the above task-oriented reasoning.

4. FEATURE/TARGET MATRICES

4.1. X from embeddings (baseline)

For each sub-ontology, we build the input matrix by stacking the embedding vectors:

$$X \in \mathbb{R}^{n \times d}, \qquad X = \text{stack}(\texttt{embeddings}).$$

This is the baseline used in all final models because it is memory-stable and does not require special encoding.

4.2. y from GO term tuples (MultiLabelBinarizer)

The targets are transformed using `MultiLabelBinarizer`:

$$y \in \{0, 1\}^{n \times |y|},$$

where $y_{ij} = 1$ indicates that protein i is annotated with the term GO j.

We also save each fitted binarizer (CC/MF/BP) to disk. The reason is reproducibility: once labels are mapped to columns, we must keep the exact same mapping to interpret predictions and to evaluate consistently across runs.

5. TRAIN/VALIDATION/TEST SPLIT

5.1. Why we need three splits

Our workflow uses validation for *decision making* (early stopping, model selection, and threshold tuning). If we reported the validation numbers, we would overestimate generalization because the validation set is implicitly "used" by our choices. Therefore, a separate test set is necessary to provide an unbiased final estimate.

5.2. The exact split implemented in code

For each ontology, the split is performed in two steps (with a fixed seed for reproducibility):

```
# Step 1: keep test untouched
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42
)

# Step 2: split the remaining 90% into train and validation
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=(0.2/0.9), random_state=42
)
```

What ratios this produces. Step 1 gives 10% as a test. Step 2 sets the validation size to `0.2/0.9` of the remaining 90%, i.e., exactly 20% of the original data set. The resulting split is, therefore:

$$\text{train} = 70\%, \quad \text{validation} = 20\%, \quad \text{test} = 10\%.$$

Why this is still a defensible choice. A larger validation set is helpful under extreme imbalance: for many rare GO terms, a 10% validation split may contain too few (or zero) positives, making both early stopping and threshold calibration unstable. Using a validation set of 20% increases the chance that the validation signal contains enough positive examples to tune the model without touching the test set.

Retrospective

We kept test at 10% to preserve an unbiased final check.

Why a two-step split instead of one call. A single `train\_test\_split` cannot directly create three splits with exact global proportions. The two-step split guarantees that the test is never reused and that the validation fraction is exact relative to the full dataset.

6. MODELS

Due to the nature of the task and the high dimensionality of the feature space, our architectural choices were severely constrained (for example random forests, or CatBoost models are not ideal for training in this kind of scenario), so we shifted to multi-label (very wide) MLPs, with sigmoid outputs (GO terms are not mutually exclusive, so softmax would be incorrect. Sigmoid returns one probability per label) and binary cross-entropy as loss. The shared design assumptions are as follows:

- embeddings already encode rich sequence information,
- we need enough capacity to map 1024-dimensional vectors to a large label space,
- regularization is essential to avoid overfitting frequent labels and ignoring rare ones.

6.1. NN1: wide–bottleneck–wide (feature filtering)

NN1 uses three dense layers (2048 → 1024 → 2048) with dropout (0.4/0.3/0.2). The middle layer is a bottleneck.

Why the bottleneck is there. The bottleneck forces compression and can behave as an implicit learned filter: the network cannot carry all input directions forward, so it must retain combinations that are most predictive for the label space.

6.2. NN2: very wide two-layer network

NN2 expands its capacity with two large hidden layers (8192 → 4096) and dropout (0.5/0.3). The intent is to test whether raw capacity alone improves the mapping from embeddings to many GO labels.

6.3. NN3: wide network with stronger regularization and higher learning rate

NN3 uses 4096 → 4096 with dropout (0.5/0.4) and a slightly higher learning rate. The idea is: if wide models are slow or unstable to optimize, a controlled learning-rate increase can help convergence, while stronger dropout counters overfitting.

Retrospective

We kept architectures "simple but big" on purpose: the main uncertainty was not the functional form (MLPs are standard) but how much capacity/regularization is needed to avoid collapsing onto the most frequent GO terms.

7. TRAINING STRATEGY

7.1. Loss and monitored metrics

We optimize binary cross-entropy and track:

- **AUC-PR** (precision–recall AUC), useful in imbalanced settings and closer in spirit to CAFA's threshold-based $F_{\max}$ PR evaluation.
- **Macro-F1**, **Micro-F1** and **Weighted-F1** at a fixed threshold (0.5) during training, mainly as a signal.

7.2. Early stopping on validation macro-F1

We use early stopping with `monitor='val\_f1\_macro'` and patience of 10 epochs, restoring the best weights.

Why macro-F1 for early stopping. Macro-F1 weighs each label equally. Under imbalance, monitoring only loss or weighted metrics can hide the fact that minority labels are ignored.

7.3. Model checkpointing

We save the best model according to the macro-F1 validation:

$$\text{best model} = \arg\max \text{val } F_{1,\text{macro}}.$$

This aligns training selection with our main objective under imbalance.

7.4. Why `ReduceLROnPlateau` was not used in the final runs

A learning-rate reduction callback was tested, but commented out because it tended to lock models into suboptimal plateaus in our runs. Given early stopping and fixed learning rates, we prioritized a simpler and more predictable training schedule.

8. EVALUATION PROTOCOL

8.1. Why threshold optimization is necessary

In multi-label settings, probabilities must be converted to label sets. A default threshold of 0.5 is arbitrary and rarely optimal under imbalance. Therefore, we calibrate a threshold on validation and apply it to the test.

8.2. Threshold search used in practice

For each trained model, we:

1. predict probabilities on validation,
2. scan a grid of thresholds (e.g. 0.10 to 0.55 with step 0.05),
3. select the threshold that maximizes validation macro-F1,
4. apply that threshold to test probabilities.

Retrospective

We intentionally do *not* optimize the threshold on test: even though it would give higher numbers, it would also destroy the point of having a clean generalization estimate.

8.3. Metric definitions

Per-label precision, recall, and F1. For each label ℓ, we define true positives (TP_ℓ), false positives (FP_ℓ) and false negatives (FN_ℓ). Then:

$$\text{Precision}_\ell = \frac{TP_\ell}{TP_\ell + FP_\ell}, \qquad \text{Recall}_\ell = \frac{TP_\ell}{TP_\ell + FN_\ell}, \tag{1}$$

$$F_{1,\ell} = \frac{2\,\text{Precision}_\ell\,\text{Recall}_\ell}{\text{Precision}_\ell + \text{Recall}_\ell}. \tag{2}$$

Macro-F1. Let L be the number of labels. Macro-F1 is the unweighted average of the per-label F1 scores:

$$F_{1,\text{macro}} = \frac{1}{L}\sum_{\ell=1}^{L} F_{1,\ell}. \tag{3}$$

Each label contributes equally, so performance on rare terms is not overshadowed by frequent ones.

Weighted-F1. Let n_ℓ be the support of label ℓ (i.e., the number of positive examples for that label). Weighted-F1 is:

$$F_{1,\text{weighted}} = \sum_{\ell=1}^{L}\left(\frac{n_\ell}{\sum_{j=1}^{L} n_j}\right) F_{1,\ell}. \tag{4}$$

This variant gives more influence to labels that appear more often in the dataset.

9. APPROACHES ATTEMPTED

9.1. InterPro augmentation: why it was attractive

InterPro domains are interpretable and biologically grounded, so including them felt like a principled way to inject structured information beyond embeddings.

9.2. Why the dense encoding failed (RAM bottleneck)

A straightforward multi-hot encoding of domains creates a huge binary matrix. This is why this approach was explicitly marked as not RAM-feasible.

9.3. Sparse strategy as a solution

We explored sparse encodings and batch-wise densification (only converting mini-batches to dense tensors). This approach is conceptually sound: it pays only for nonzero entries. However, it requires custom data generators and careful handling of sparse formats. Within project time constraints, the baseline embedding-only pipeline remained the primary path for stable runs.

9.4. 10-fold cross-validation

We implemented a K-Fold training routine to reduce dependence on a single split. In practice, the computational cost was high and we did not observe consistent improvements compared to the holdout protocol. Given that our pipeline already allocates a substantial validation set for robust tuning, we prioritized the simpler split-based evaluation.

Retrospective

Cross-validation is attractive in theory, but under heavy models and high-dimensional multi-label targets, it can become expensive while adding limited insight beyond a careful train/validation/test setup.

10. RESULTS

This section summarizes the final test-set performance of the three neural baselines (NN1-NN2-NN3) across the three GO namespaces (CC, MF, BP). For each model and ontology we report:

- **AUC-PR (micro)** on the test set;
- **F1** in macro, micro, and weighted variants on the test set;
- the **optimal decision threshold** τ^* selected on the validation set and then applied once to the test set.

10.1. How to read these numbers

Because GO annotation is a highly imbalanced multi-label task, each metric captures a different view:

- **AUC-PR (micro)**: summarizes the precision–recall trade-off across thresholds, pooling decisions globally. It is typically more informative than ROC-AUC in imbalanced settings because it focuses on the positive class.
- $F_{1,\text{micro}}$: computed from globally aggregated $TP/FP/FN$ over all labels. It tends to reflect performance on frequent terms and the overall quality of label assignment.
- $F_{1,\text{weighted}}$: average of per-label F1 weighted by label prevalence (support). It remains close to micro-F1 but still preserves a per-label formulation.
- $F_{1,\text{macro}}$: unweighted average over labels, giving the same importance to rare and frequent GO terms. This is the most sensitive indicator for minority labels and the most conservative summary in long-tail settings.

Why thresholds matter. The models output probabilities, but predictions require a threshold. A default $\tau = 0.5$ is arbitrary and often suboptimal under imbalance; therefore we select τ^* on validation to maximize macro-F1 and then keep it fixed for test reporting.

10.2. Cellular Component (CC)

Table 1. CC – Test results (using validation-optimized threshold τ^*).

Model	τ^*	AUC-PR (micro)	$F_{1,\text{macro}}$	$F_{1,\text{micro}}$	$F_{1,\text{weighted}}$
NN1	0.35	0.7793	0.4120	0.7104	0.7090
NN2	0.35	0.7793	0.4263	0.7114	0.7111
NN3	0.45	0.7783	0.4222	0.7128	0.7103

CC: all models are extremely close; NN2 yields the best macro-F1, while NN3 slightly leads micro-F1.

Interpretation (CC). CC shows consistently high global performance (micro AUC-PR ≈ 0.78, micro/weighted F1 ≈ 0.71) across all architectures. Differences are marginal, suggesting that for CC the embedding representation already captures most of the learnable signal and the choice of architecture mainly affects fine-grained trade-offs (slightly better macro-F1 vs slightly better micro-F1).

10.3. Molecular Function (MF)

Table 2. MF – Test results (using validation-optimized threshold τ^*).

Model	τ^*	AUC-PR (micro)	$F_{1,\text{macro}}$	$F_{1,\text{micro}}$	$F_{1,\text{weighted}}$
NN1	0.40	0.7752	0.5654	0.7235	0.7139
NN2	0.30	0.7796	0.5713	0.7182	0.7150
NN3	0.50	0.7796	0.5728	0.7232	0.7116

MF: best overall scores; NN3 achieves the best macro-F1, while NN1 and NN3 are essentially tied on micro-F1.

Interpretation (MF). MF is the strongest ontology overall (micro AUC-PR close to 0.78 and micro-F1 around 0.72). The three models behave very similarly: NN3 achieves the best macro-F1 (rare-label sensitivity), while NN1 remains competitive and even slightly better on micro-F1 (0.7235 vs 0.7232 for NN3). This reinforces the idea that once embeddings provide a strong representation, architecture changes tend to produce fewer refinements rather than dramatic improvements.

10.4. Biological Process (BP)

Table 3. BP – Test results (using validation-optimized threshold τ^*).

Model	τ^*	AUC-PR (micro)	$F_{1,\text{macro}}$	$F_{1,\text{micro}}$	$F_{1,\text{weighted}}$
NN1	0.25	0.5073	0.3438	0.5014	0.4883
NN2	0.35	0.5118	0.3652	0.5089	0.4984
NN3	0.40	0.5117	0.3573	0.5065	0.4942

BP: the hardest ontology; NN2 provides the clearest gain, especially in macro-F1 (minority-label performance).

Interpretation (BP). BP is clearly the hardest label space (micro AUC-PR ≈ 0.51 and macro-F1 ≈ 0.35), which matches the expectation of a larger, more imbalanced and more diverse ontology. Here, the effect of model capacity is more visible: NN2 yields the highest macro-F1 (0.3652) and improves micro/weighted F1 as well. In other words, BP is the setting where "more capacity" can help capture additional structure in the long tail.

10.5. Overall picture

Across CC and MF, performance differences are small and mostly within a narrow band, while BP shows a more meaningful separation in favor of NN2. This pattern is consistent with the intuition that the hardest ontology is the one that benefits most from additional capacity and expressiveness.

10.6. CAFA Evaluator

As a final step, we applied the **CAFA evaluator** to our predictions. Concretely, we generated a CAFA-formatted prediction file and ran the evaluator to obtain the *precision–recall coordinates* across thresholds for each GO namespace (BP, CC, MF). Using the coordinates returned by the evaluator, we produced the two summary plots below: the PR curves and a compact comparison of CAFA metrics.

Brief metric interpretation. The CAFA evaluator reports two key scalar summaries:

- F_{max}: the maximum F-score achievable over all thresholds. Higher is better. It captures the best precision–recall trade-off obtainable by selecting an optimal threshold.
- S_{min}: a semantic distance score minimized over thresholds (lower is better). It combines two components: penalties for *missed true annotations* (false negatives) and for *incorrect predicted annotations* (false positives), taking into account the ontology structure (errors on more informative terms typically weigh more).

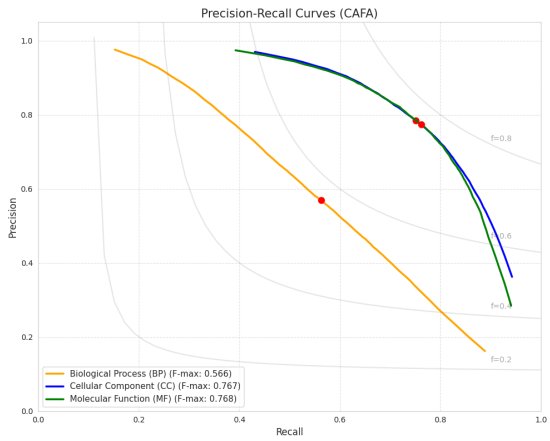


Figure 4. Precision–Recall curves computed with the CAFA evaluator for BP, CC, and MF. Each curve summarizes the trade-off across thresholds; the highlighted point corresponds to the threshold achieving F_{max}.

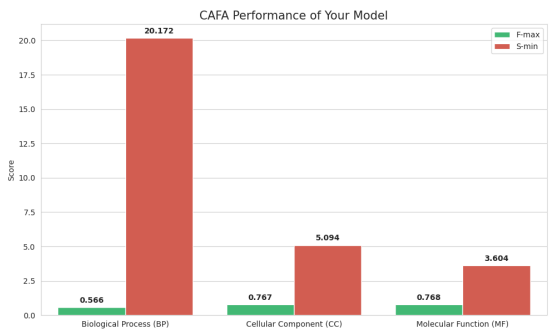
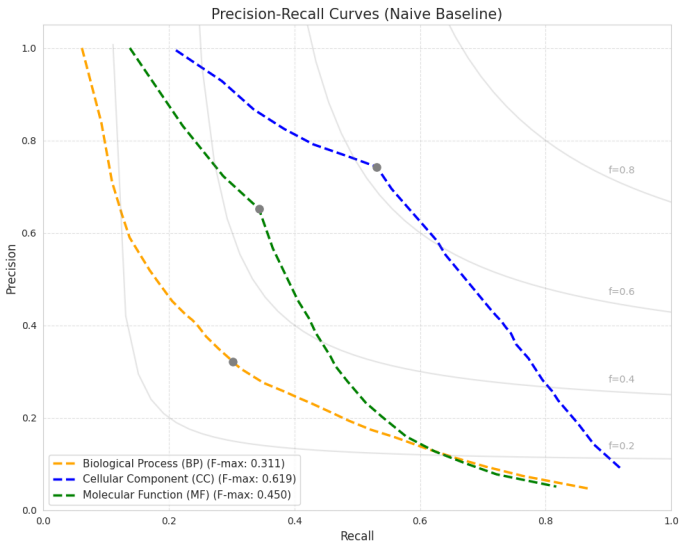


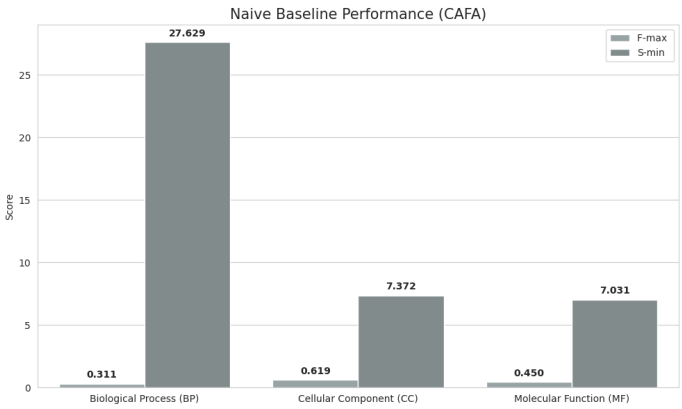
Figure 5. CAFA summary metrics per ontology: F_{max} (higher is better) and S_{min} (lower is better).

10.7. Naive baseline

As an additional sanity check, we implemented a simple **naive baseline**. The goal is not to compete with the neural models, but to provide a lower bound for CAFA-style evaluation. The baseline produces a CAFA-formatted prediction file and is evaluated with the same CAFA pipeline.



(a) Naive baseline PR curves (CAFA).



(b) Naive baseline: $F_{\max}$ and $S_{\min}$ (CAFA).

Figure 6. CAFA evaluation of the naive baseline.

10.8. Comparison with the naive and IPR baseline

Finally, we compared our final model against the naive and IPR baseline under the CAFA evaluator. The comparison is reported both in terms of precision–recall curves and in terms of $F_{\max}$, highlighting the improvement obtained by a learned model over a simple heuristic baseline. *Notably, the gap is largest in BP, confirming that the learned model captures non-trivial signal in the most challenging ontology.*

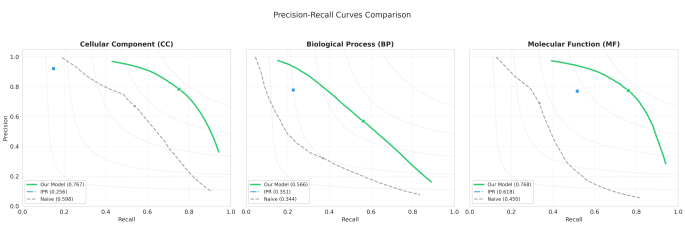


Figure 7. Precision–Recall comparison between our model and the naive and IPR baseline for CC, BP and MF.

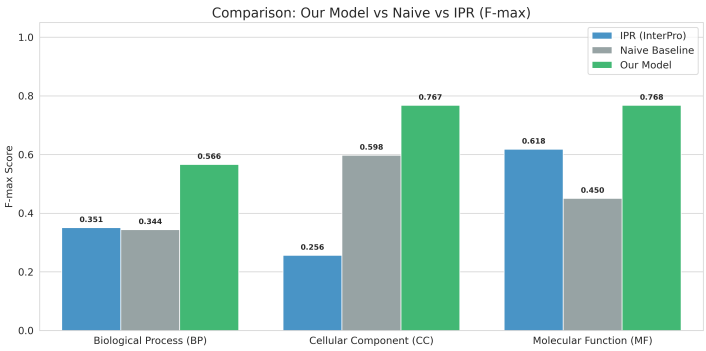


Figure 8. $F_{\max}$ comparison between our model and the naive and IPR baseline across the three ontologies.

10.9. Hierarchical Consistency and Prediction Correction

In addition to generating the initial submission, a post-processing phase was implemented to ensure the biological consistency of the results according to the structure of the Gene Ontology's Directed Acyclic Graphs (DAGs). This procedure, described and implemented in the `Project results.ipynb` notebook, is useful for aligning the model's numerical output with biological constraints.

Initially, compliance with the hierarchical rule was verified, according to which a "child" term must always have a probability less than or equal to that of its "parent" terms. Finding numerous violations of this constraint in the raw predictions provided by neural models, we developed a dedicated correction function that acts according to two main logics:

- **Ancestor Recovery:** Automatic addition of all missing parent terms for each term predicted with a probability above the threshold.
- **Probability Propagation:** Correction of the probabilities of parent terms already present, setting them to the value $P(\text{parent}) = \max(P(\text{parent})_{\text{orig}}, P(\text{child}))$, thus ensuring that the probability does not decrease when tracing back up the graph (*propagation*).

This intervention led to the creation of a second output file, named `submission_with_hierarchy_enforced.tsv`, fully compliant with the GO hierarchy. Although this operation theoretically should results in a slight decrease in pure precision due to the increase in the number of predicted terms, the significant increase in *Recall* and the structural correction of probabilities lead to an overall improvement in the final score and biological robustness of the model. (Anyway both submission files are provided with the expectation that the model yielding the best results will be the one evaluated, PLEASE :)).

10.10. Submission Strategy and Thresholding

For the generation of the final submission, we decided to include all predictions with a probability greater than 1% ($P > 0.01$). This choice was dictated by the evaluation methodology of the *CAFA evaluator*, which analyzes models by applying a sliding scale of confidence thresholds to calculate key metrics such as the Precision-Recall curve and the F_{max} score. Maintaining a large set of predictions, even with low confidence, proved to be the most sensible choice to allow the evaluator to correctly reconstruct the model's performance across the entire sensitivity spectrum, thus optimising the final score based on the hierarchical structure of the Gene Ontology.

11. CONCLUSION

Overall, the three architectures deliver *comparable* test performance in CC and MF, while BP benefits more from increased capacity. From a purely performance-driven standpoint:

- **CC:** NN2 achieves the best macro-F1, NN3 slightly leads micro-F1 (differences are minimal).
- **MF:** NN3 achieves the best macro-F1, while NN1 and NN3 are essentially tied to micro-F1.
- **BP:** NN2 provides the clearest improvement, especially in macro-F1, indicating a better coverage of rarer terms.

However, model selection is not only about the last decimal point: computational cost matters. NN2 (and to a lesser extent NN3) uses much wider hidden layers than NN1, implying larger matrix multiplications and more parameters, hence a higher training and tuning cost. Given that gains are marginal in two ontologies (CC and MF), **NN1 is the most sensible default compromise** between training efficiency and predictive performance.

A pragmatic strategy is, therefore,:

- use **NN1** as the default model when training time/resources are constrained and when performance across ontologies must be balanced;
- optionally select **NN2 specifically for BP** if the primary goal is to maximize macro-F1 on the most challenging ontology and additional compute is acceptable.

Retrospective

The main takeaway is that embeddings already provide a strong representation: once that baseline is in place, architectural changes, in the realm of wide NN, produce mostly incremental improvements. In this regime, the most reliable “winner” is often the model that offers a better compute–performance trade-off rather than the absolute highest score on a single metric.